PROJECT TEAM MATCHER

Intelligent Team Formation Platform

SOFTWARE REQUIREMENTS SPECIFICATION

Course	UCS503P
Submitted to	Kritika Goel
Department	Computer Science and Engineering
Institute	Thapar Institute of Engineering and Technology, Patiala

Project Team

Mohit Garg	1024031006
Paras Sharma	1024030609
Manan Dhingra	1024030222
Parv Panwar	1024030226

Project Type	Software Engineering Project
Development Period	August – October 2026
Platform	Web Application
Department	Computer Science and Engineering

August 2026

1. Introduction

1.1 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of Project Team Matcher, a web-based platform for forming student teams at Thapar Institute of Engineering and Technology.

The document describes the expected behaviour of the system, the users who interact with it, the information maintained by the system, and the constraints that the implementation should satisfy. It is intended to provide a common reference for development, testing and evaluation.

1.2 1.2 Scope

Project Team Matcher is restricted to Thapar students. Students register and access the platform using their Thapar Student ID. Each student can maintain a profile containing a resume, phone number, GitHub profile, LinkedIn profile and skills.

The system allows students to create teams for projects, hackathons, competitions and similar collaborative activities. Team leaders specify the required number of members and required skills. Incomplete teams are visible to eligible students across campus. Students can search and filter available teams, particularly by hackathon and skills, and request to join a team.

When the required number of students has joined a team, the team is considered complete and becomes private. Team members can continue to access the project through their dashboard.

1.3 1.3 Intended Users

The primary users are Thapar students. A student may act as a regular team member, a team leader, or both at different times.

- **Student:** Maintains a profile, discovers teams and sends join requests.
- **Team Leader:** Creates teams, specifies requirements and manages join requests.

2. Overall Description

2.1 2.1 Product Perspective

The system is a standalone web application consisting of a frontend, backend API and relational database. The frontend provides the student interface, while the backend is

responsible for authentication, authorization, validation, team management and database operations.

2.2 2.2 Product Functions

The major functions of the system are:

- Thapar Student ID based registration and login.
- Student profile creation and management.
- Resume upload and professional profile links.
- Team creation.
- Team size and skill requirement definition.
- Available team discovery.
- Hackathon and skill based filtering.
- Join request submission.
- Join request acceptance or rejection by team leaders.
- Team capacity validation.
- Automatic transition of completed teams to private status.
- Dashboard for current projects and available teams.

2.3 2.3 Operating Environment

The system is intended to operate as a browser-based web application. Users access it through modern desktop or mobile web browsers. The frontend uses HTML5, CSS3 and Vanilla JavaScript without external frontend frameworks. The backend will be implemented using Node.js and Express.js and will communicate with a MySQL relational database.

2.4 2.4 Design Constraints

- Access must be restricted to Thapar students.
- Student identity must be associated with a valid Student ID.
- Important validation must be performed on the backend.
- Team capacity must not be exceeded.

- Completed teams must not remain publicly joinable.
- The frontend shall use only HTML5, CSS3 and Vanilla JavaScript.
- The frontend shall not use React, Next.js, TypeScript, Tailwind CSS, Bootstrap or any other external frontend framework.
- The frontend code shall be structured so mock JavaScript data can later be replaced by Node.js/Express.js API calls.
- The system should remain simple enough to be implemented and tested within the academic project period.

3. User Requirements

3.1 3.1 Student Registration and Login

Students shall provide their Thapar Student ID to access the system. The system shall validate the Student ID according to the authentication mechanism used by the implementation.

After successful authentication, the student shall be able to access the dashboard and profile.

3.2 3.2 Student Profile

A student shall be able to add and update:

- Resume
- Phone number
- GitHub profile
- LinkedIn profile
- Skills

Profile information may be viewed by other students where required for team discovery, subject to the system's privacy and authorization rules.

3.3 3.3 Dashboard

The dashboard shall provide access to:

- Create a Team

- Join a Team
- Current Working Projects
- Available Teams to Join

3.4 3.4 Team Creation

A team leader shall be able to create a team by entering:

- Project or hackathon name
- Project description
- Required number of students
- Required skills

The created team shall initially have an open status and shall be discoverable by eligible students.

3.5 3.5 Team Discovery

Students shall be able to view teams that are currently open for membership. Each team listing should provide sufficient information to help the student decide whether to request membership.

The system shall support filters based on:

- Hackathon or project
- Required skills

3.6 3.6 Join Requests

A student shall be able to send a request to join an open team. The system shall record the request and prevent inappropriate duplicate requests.

A team leader shall be able to accept or reject requests. Acceptance shall only succeed if the team still has an available position.

3.7 3.7 Team Completion

The system shall maintain the current number of team members. When the number of accepted members reaches the required team size, the team shall automatically be marked as completed/private.

A private team shall not appear as an available team for new students. Existing members shall continue to access it from their current working projects.

4. Functional Requirements

FR-01	The system shall allow a student to register or authenticate using a valid Thapar Student ID.
FR-02	The system shall prevent unauthorized users from accessing student-only functionality.
FR-03	The system shall allow students to create and update their profiles.
FR-04	The system shall allow students to upload a resume.
FR-05	The system shall allow students to store phone, GitHub and LinkedIn details.
FR-06	The system shall allow students to add and manage skills.
FR-07	The system shall display a dashboard containing Create a Team, Join a Team, Current Working Projects and Available Teams.
FR-08	The system shall allow a student to create a team.
FR-09	The system shall allow a team leader to define the required team size.
FR-10	The system shall allow a team leader to define required skills.
FR-11	The system shall display incomplete teams to eligible students.
FR-12	The system shall allow students to filter available teams by hackathon/project and skills.
FR-13	The system shall allow a student to submit a request to join an open team.
FR-14	The system shall allow a team leader to accept or reject join requests.
FR-15	The system shall prevent a team from exceeding its configured capacity.
FR-16	The system shall prevent new join requests from being accepted after a team is completed.
FR-17	The system shall mark a team private when its required capacity is reached.
FR-18	The system shall display a student's accepted projects under Current Working Projects.
FR-19	The system shall allow team members to access private team information according to their authorization.

5. External Interface Requirements

5.1 5.1 User Interface

The user interface shall provide a simple dashboard-oriented experience. The primary navigation shall allow students to move between profile management, team creation, team discovery, join requests and current projects.

Team listings should clearly show project/hackathon information, required skills, current team size and remaining positions.

5.2 5.2 Software Interface

The frontend shall communicate with the backend through REST APIs. The initial frontend may use mock JavaScript data during development, but its structure shall allow those mock operations to be replaced cleanly by API calls without redesigning the user interface.

The backend shall communicate with the relational database for persistent storage. Database access shall not be exposed directly to the browser.

5.3 5.3 File Interface

The system shall support resume upload. Uploaded files shall be validated for acceptable file type and size before storage.

6. Non-Functional Requirements

6.1 6.1 Performance

Search, filtering, dashboard loading and team operations should respond efficiently under normal usage. Database indexes should be used for frequently queried attributes such as Student ID, team status, project/hackathon and skills.

6.2 6.2 Scalability

The application should be capable of supporting a large number of Thapar students and teams. The frontend, backend and database should be separated so that individual components can be optimized or scaled independently if required.

6.3 6.3 Reliability

The system shall maintain consistent team membership data. Concurrent requests must not cause a team to exceed its required capacity.

6.4 6.4 Security

The system shall enforce authentication and authorization on the backend. Students shall not be able to modify resources belonging to other students or teams without permission.

6.5 6.5 Usability

A student should be able to discover a suitable team and submit a join request using a small number of clear steps. Team status and remaining positions should be easy to understand.

6.6 6.6 Maintainability

The application should use modular frontend and backend components with clear separation of responsibilities. APIs and database operations should follow consistent conventions.

7. Technology Stack

The implementation shall use the following technology stack:

Area	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	Node.js, Express.js
Database	MySQL
API	REST API
Authentication	Secure sessions / JWT
Testing	Jest, Postman
Version Control	Git, GitHub
Deployment	Docker / cloud host

The frontend shall use HTML5, CSS3 and Vanilla JavaScript. React, Next.js, TypeScript, Tailwind CSS, Bootstrap and other external frontend frameworks shall not be used. The frontend shall be structured cleanly so that mock JavaScript data can later be replaced with API calls to the Node.js and Express.js backend.

8. Data Requirements

The main data entities are:

- **Student:** Student ID and authentication-related information.
- **StudentProfile:** Resume, phone, GitHub, LinkedIn and profile information.
- **Skill:** A reusable skill definition.
- **StudentSkill:** Association between a student and a skill.
- **Team:** Team name, project/hackathon, description, required size and status.

- **TeamSkill:** Skills required by a team.
- **TeamMember:** Confirmed membership of students in teams.
- **JoinRequest:** A student's request to join a team and its status.
- **Project/Hackathon:** Project or event information associated with a team.

The database shall maintain unique Student IDs and should prevent duplicate active team membership records where such membership is not permitted by the implementation.

9. Business Rules

1. Only eligible Thapar students may use the platform.
2. A student must have an authenticated account before creating or joining a team.
3. Every team shall have a defined maximum number of members.
4. A team shall remain available while it has open positions.
5. A join request shall not automatically guarantee membership.
6. Only the team leader or an authorized team manager may accept or reject join requests.
7. A team shall become private when its required member count is reached.
8. Private teams shall not accept new members through the public team discovery interface.
9. Backend validation shall be authoritative for team capacity and authorization.

10. Use Case Summary

Use Case	Description
Register/Login	Student authenticates using Thapar Student ID.
Manage Profile	Student adds or updates resume, contact and professional details and skills.
Create Team	Student creates a team and defines its requirements.
Browse Teams	Student views open teams across campus.
Filter Teams	Student filters teams using hackathon/project and skills.
Request to Join	Student sends a membership request to an open team.
Manage Requests	Team leader accepts or rejects student requests.
Complete Team	System marks a team private when capacity is reached.
View Projects	Student views accepted/current working projects on the dashboard.

11. Error and Exception Handling

The system shall return appropriate errors for invalid operations rather than allowing the application to fail unexpectedly.

Important cases include:

- Invalid or unauthorized Student ID.
- Missing required profile or team information.
- Invalid resume file.
- Request to join a non-existent team.
- Request to join a completed/private team.
- Duplicate join request.
- Attempt to exceed team capacity.
- Unauthorized team or profile modification.
- Database or service failure.

Errors should be logged appropriately on the server without exposing sensitive implementation details to users.

12. Testing Requirements

Testing shall cover functional correctness, security and reliability.

- **Unit Testing:** Validation, filtering and team-capacity logic.
- **API Testing:** Authentication, profile, team and join-request endpoints.
- **Integration Testing:** Frontend, backend and database workflows.
- **System Testing:** Complete registration-to-team-completion flow.
- **Security Testing:** Unauthorized access and API authorization checks.
- **Load Testing:** Concurrent dashboard, search and join-request activity.

A critical test case is when multiple students attempt to join the final available position in a team at the same time. Only requests accepted within the available capacity should result in membership.

13. Acceptance Criteria

The system will be considered acceptable when:

1. Only valid Thapar students can access the application.
2. Students can maintain the required profile information.
3. Students can create teams with required skills and team size.
4. Open teams are visible to students across campus.
5. Students can filter teams by hackathon/project and skills.
6. Students can send join requests.
7. Team leaders can manage join requests.
8. Team capacity cannot be exceeded, including during concurrent requests.
9. Completed teams automatically become private.
10. Current working projects are displayed on the dashboard.
11. The application handles invalid and unauthorized operations without crashing.
12. Core operations remain responsive under the planned test load.

14. Conclusion

Project Team Matcher is intended to provide a focused and reliable platform for Thapar students to form teams for hackathons, projects, competitions and related collaborative activities.

This SRS defines the requirements needed to implement the core system without introducing unnecessary complexity. The most important engineering requirements are controlled student access, reliable team-capacity handling, efficient team discovery, secure authorization, consistent database operations and a scalable architecture.

The requirements in this document provide the basis for system design, implementation, testing and final evaluation of the Project Team Matcher software engineering project.